

This Python script is a chatbot built using **Streamlit** for a web interface, **NLTK** for text processing, and **scikit-learn** for text classification using **TF-IDF** and **Logistic Regression**.

1. Import Required Libraries

```
python
CopyEdit
import os
import nltk
import ssl
import streamlit as st
import random
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

- **os**: For handling file paths.
 - **nltk**: A natural language processing (NLP) library.
 - **ssl**: Used for handling SSL security issues when downloading NLTK datasets.
 - **streamlit**: A Python library for building web apps.
 - **random**: To randomly select responses.
 - **TfidfVectorizer** (from `sklearn`): Converts text into numerical values.
 - **LogisticRegression** (from `sklearn`): Used for text classification.
-

2. Handle SSL and Download NLTK Data

```
python
CopyEdit
ssl._create_default_https_context = ssl._create_unverified_context
nltk.data.path.append(os.path.abspath("nltk_data"))
nltk.download('punkt')
```

- This avoids SSL verification errors when downloading NLTK data.
 - Adds a local directory for storing NLTK data.
 - Downloads the **punkt** tokenizer (used to split text into sentences and words).
-

3. Define the Chatbot's Intent Dataset

```
python
CopyEdit
intents = [
    {
        "tag": "greeting",
        "patterns": ["Hi", "Hello", "Hey", "How are you", "What's up"],
        "responses": ["Hi there", "Hello", "Hey", "I'm fine, thank you", "Nothing
much"]
    },
    {
        "tag": "goodbye",
        "patterns": ["Bye", "See you later", "Goodbye", "Take care"],
        "responses": ["Goodbye", "See you later", "Take care"]
    },
    ...
]
```

- This **intent-based dataset** consists of:
 - **tag**: A label for intent (e.g., "greeting", "goodbye").
 - **patterns**: Common ways users might phrase their inputs.
 - **responses**: Predefined chatbot replies.
-

4. Create and Train the Chatbot Model

```
python
CopyEdit
vectorizer = TfidfVectorizer()
clf = LogisticRegression(random_state=0, max_iter=10000)
```

- **TF-IDF Vectorizer** converts user inputs into numerical representations.
- **Logistic Regression** is the classifier that predicts the intent.

4.1 Prepare Training Data

```
python
CopyEdit
tags = []
patterns = []
for intent in intents:
    for pattern in intent['patterns']:
        tags.append(intent['tag'])
        patterns.append(pattern)
```

- Extracts all possible user **patterns** and their **corresponding intent tags**.

4.2 Train the Model

```
python
CopyEdit
x = vectorizer.fit_transform(patterns) # Convert text to numerical form
y = tags # Labels
clf.fit(x, y) # Train the classifier
```

- Converts **patterns** into a numerical TF-IDF matrix.
 - Trains the **Logistic Regression classifier** with input-output pairs.
-

5. Define the Chatbot Response Function

```
python
CopyEdit
def chatbot(input_text):
    input_text = vectorizer.transform([input_text]) # Convert user input into TF-IDF
    format
    tag = clf.predict(input_text)[0] # Predict intent
    for intent in intents:
        if intent['tag'] == tag:
            response = random.choice(intent['responses']) # Pick a random response
            return response
```

- Takes user input.
- Converts it into TF-IDF form.
- Uses the trained classifier to predict the **intent tag**.

- Finds the matching intent in the dataset and selects a random **response**.
-

6. Create a Web App Using Streamlit

```
python
CopyEdit
counter = 0
```

- A global **counter** is used to track different user inputs.

6.1 Define the Main Function

```
python
CopyEdit
def main():
    global counter
    st.title("Chatbot")
    st.title("Created by Avanthi Institute of Engineering and Technology")
    st.write("Welcome to the chatbot. Please type a message and press Enter to start the conversation.")
```

- Displays the **chatbot title** and **introduction message**.

6.2 Get User Input

```
python
CopyEdit
counter += 1
user_input = st.text_input("You:", key=f"user_input_{counter}")
```

- Uses `st.text_input()` to allow users to enter messages.
- Each input is assigned a unique **key** based on the counter.

6.3 Generate Chatbot Response

```
python
CopyEdit
if user_input:
    response = chatbot(user_input) # Get chatbot response
    st.text_area("Chatbot:", value=response, height=100, max_chars=None,
key=f"chatbot_response_{counter}")
```

- If the user enters text, the chatbot processes the input and generates a response.
- `st.text_area()` displays the chatbot's reply.

6.4 Exit Condition

```
python
CopyEdit
if response.lower() in ['goodbye', 'bye']:
    st.write("Thank you for chatting with me. Have a great day!")
    st.stop()
```

- If the chatbot responds with "goodbye" or "bye", the app stops.
-

7. Run the Web App

```
python  
CopyEdit  
if __name__ == '__main__':  
    main()
```

- Ensures `main()` runs when the script is executed.
-

How the Chatbot Works

1. The user enters a message.
 2. The text is **vectorized** using TF-IDF.
 3. The **trained Logistic Regression model** predicts the intent.
 4. A **random response** is selected from the matched intent.
 5. The response is displayed in the Streamlit web interface.
-

Summary

- **Intent-based chatbot** using TF-IDF and Logistic Regression.
- **Training data** consists of predefined intents.
- **Text vectorization** converts user inputs into numerical format.
- **Logistic Regression model** predicts user intent.
- **Streamlit UI** provides a simple chat interface.

Let me know if you need further clarification! 🚀